

Krishna_Karthik_Peri_Midterm.ino

```
1  /*
2  * =====
3  * File: Midterm_Cumulative_20250626_CMNTD.ino
4  * Author: Krishna Karthik, Peri
5  * Date Modified: 26.06.2025
6  * Purpose: Autonomous Robotics System, Midterm Exam
7  * Description:
8  *   Arduino sketch for a smart skateboard controller featuring:
9  *     - Dual control modes: accelerometer-based and joystick-based
10 *     - DC motor speed control with safety lock on fall detection
11 *     - Stepper motor steering control
12 *     - 7-segment display via shift register for speed feedback
13 *     - Servo-based analog speedometer
14 *     - Automatic and manual headlight LED brightness control
15 *     - Obstacle detection and automatic braking using ultrasonic sensor
16 *     - Comprehensive startup and test routines
17 *
18 * Pin Selection Rationale:
19 *   - PWM-capable pins (3, 10, 11) are used for DC motor (analogWrite), servo (Servo
20 library), and headlight LED (PWM via shift register OE).
21 *   - Stepper motor pins (1, 2, 4, 12) are chosen to match the excitation coil order
22 required by the Stepper.h library for correct stepping sequence.
23 *   - Analog pins (A0-A3) are used for sensors and joystick axes for high-resolution
24 analog input.
25 *   - Digital pins 7-9, 11 are used for 74HC595 shift register control, as these pins are
26 free and support fast digitalWrite operations.
27 *   - Pin 13 is used for the joystick button, leveraging the onboard pull-up resistor.
28 *   - I2C communication with the ADXL345 accelerometer uses dedicated SDA/SCL pins
29 (handled by Wire.h).
30 *
31 * Stepper Motor Initialization:
32 *   The Stepper object is initialized as:
33 *     Stepper(stepsPerRevolution, IN1, IN2, IN3, IN4)
34 *   The order of hBridgeInput1, hBridgeInput2, hBridgeInput3, hBridgeInput4 matches the
35 coil excitation order required by the Stepper.h library for correct rotation.
36 *
37 * Required Libraries:
38 *   - Servo.h: For controlling the analog speedometer servo
39 *   - Stepper.h: For stepper motor control (steering)
40 *   - Wire.h: For I2C communication with the ADXL345 accelerometer
41 *
42 * Additional Notes:
43 *   - The code is modular, with each function responsible for a specific subsystem.
44 *   - All hardware initialization and safety checks are performed in setup().
45 *   - The main loop handles real-time control, display, and safety logic.
46 *   - All pin assignments and hardware dependencies are documented for easy hardware
47 mapping and troubleshooting.
48 * =====
49 */
50
51 // ----- Pin Configuration -----
52 // H-Bridge motor driver pins for stepper motor control
```

```

46 #define hBridgeInput1 12    // IN1 for stepper motor
47 #define hBridgeInput2 4     // IN2 for stepper motor
48 #define hBridgeInput3 2     // IN3 for stepper motor
49 #define hBridgeInput4 1     // IN4 for stepper motor
50
51 // Ultrasonic sensor pins
52 #define ultraSonicEcho 5     // Echo pin for ultrasonic distance sensor
53 #define ultraSonicTrigger 6 // Trigger pin for ultrasonic distance sensor
54
55 // 74HC595 Shift Register pins for 7-segment display and headlight LED
56 #define shiftRegisterInput 7 // Serial data input (DS)
57 #define shiftRegisterOutputRegisterClock 8 // Latch clock (ST_CP)
58 #define shiftRegisterShiftClock 9 // Shift clock (SH_CP)
59 #define shiftRegisterEnable 11 // Output enable (OE), also used for
headlight LED PWM
60
61 // Servo motor pin for speedometer
62 #define servoMotorSignal 10 // Servo signal pin
63
64 // Joystick button pin
65 #define joystickButton 13 // Joystick push button (active LOW)
66
67 // I2C address for ADXL345 accelerometer
68 #define ADXL345 0x53
69
70 // 7-segment display segment mapping (to shift register outputs)
71 #define displaySegmentA 0
72 #define displaySegmentB 1
73 #define displaySegmentC 2
74 #define displaySegmentD 3
75 #define displaySegmentE 4
76 #define displaySegmentF 5
77 #define displaySegmentG 6
78 #define displaySegmentH 7 // Decimal point (not used)
79
80 // Analog pins for joystick and sensors
81 #define joyStickThumbGripX A2 // Joystick X-axis
82 #define joyStickThumbGripY A3 // Joystick Y-axis
83 #define headLightPotentiometer A0 // Potentiometer for headlight brightness
84 #define headPhotoResistor A1 // Photoresistor for ambient light
85
86 // DC motor PWM output pin
87 #define dcMotorOutput 3
88
89 #include <Servo.h>
90 #include <Stepper.h>
91 #include <Wire.h>
92
93 // ----- Global Variables and Constants -----
94
95 // Control mode flags
96 bool isRemoteControlled = false; // false: accelerometer, true: joystick
97 bool isMasterLocked = false; // true: system locked due to fall
98 bool lastJoyStickButtonState = HIGH; // For button debounce

```

```

99
100 // 7-segment display segment order mapping for shift register
101 char registerOutputSevenSegmentDisplayMap[7] = {displaySegmentC, displaySegmentD,
displaySegmentE, displaySegmentG, displaySegmentF, displaySegmentA, displaySegmentB}; //
Register: Output 8 -> Output 2
102
103 // Accelerometer calibration offsets
104 const int accelerometerXOffset = -2;
105 const int accelerometerYOffset = -1;
106 const int accelerometerZOffset = 1;
107
108 // Motor speed and control constants
109 const int brakingFactor = 1;
110 const int maximumMotorSpeed = 255;
111 const int maximumSpeedIncrement = 8;
112 const int minimumMotorSpeed = 155;
113 const int minimumSpeedIncrement = 1;
114
115 // Stepper motor configuration
116 const int stepperMotorStepsPerRevolution = 2048;
117 const int stepperMotorSpeed = 10;
118
119 // Accelerometer thresholds
120 const float accelerationXMaximum = 2.00;
121 const float accelerationXThreshold = 0.35;
122 const float emergencyBrakeThreshold = 15.0;
123 const float gForceDeadzoneMinimum = 0.3;
124 const float gForceDeadzoneMaximum = 1.5;
125
126 // Joystick thresholds
127 const float joyStickThumbGripXThreshold = 30;
128
129 // Ultrasonic sensor threshold
130 const float ultraSoundObstacleDistanceThreshold = 35.0;
131
132 // Variables for sensor readings and control
133 float accelerationX, accelerationY, accelerationZ;
134 float motorSpeed, motorSpeedPercentage;
135 int headlightLED = HIGH;
136 int headLightBrightness;
137 int lastStepperMotorPosition = 0;
138 int joyStickThumbGripXReading, joyStickThumbGripYReading;
139 int preObstacleSpeedPercentage;
140
141 // 7-segment display digit to segment mapping
142 int sevenSegmentDisplayNumberMap[10][7] = {
143     // a,b,c,d,e,f,g -> 7 segment
144     {1,1,1,1,1,1,0}, //0
145     {0,1,1,0,0,0,0}, //1
146     {1,1,0,1,1,0,1}, //2
147     {1,1,1,1,0,0,1}, //3
148     {0,1,1,0,0,1,1}, //4
149     {1,0,1,1,0,1,1}, //5
150     {1,0,1,1,1,1,1}, //6

```

```

151     {1,1,1,0,0,0,0}, //7
152     {1,1,1,1,1,1,1}, //8
153     {1,1,1,1,0,1,1} //9
154 };
155
156 // Servo and stepper motor objects
157 Servo speedometerServo;
158 Stepper stepperMotor = Stepper(stepperMotorStepsPerRevolution, hBridgeInput1,
159 hBridgeInput2, hBridgeInput3, hBridgeInput4);
160
161 // Debounce timing variables
162 unsigned int headLightLedPotentiometerDebounceTime = 0;
163 unsigned int headLightLedPotentiometerDelay = 200;
164 unsigned int joyStickButtonDebounceDelay = 200;
165 unsigned int joyStickButtonDebounceTime = 0;
166 unsigned int joyStickThumbGripDebounceDelay = 200;
167 unsigned int joyStickThumbGripXDebounceTime = 0;
168 unsigned int joyStickThumbGripYDebounceTime = 0;
169
170 // ----- Arduino Setup -----
171 /*
172  * Initializes all hardware, calibrates sensors, and performs startup checks.
173  */
174 void setup(){
175     // Pin modes
176     pinMode(dcMotorOutput, OUTPUT);
177     pinMode(ultraSonicEcho, INPUT);
178     pinMode(ultraSonicTrigger, OUTPUT);
179     pinMode(shiftRegisterInput, OUTPUT);
180     pinMode(shiftRegisterOutputRegisterClock, OUTPUT);
181     pinMode(shiftRegisterShiftClock, OUTPUT);
182     pinMode(shiftRegisterEnable, OUTPUT);
183     pinMode(servoMotorSignal, OUTPUT);
184     pinMode(joystickButton, INPUT_PULLUP);
185
186     digitalWrite(dcMotorOutput, HIGH); // Ensure motor is off at start
187
188     speedometerServo.attach(servoMotorSignal); // Attach servo
189     stepperMotor.setSpeed(10); // Set stepper speed
190
191     Serial.begin(9600); // Serial for debugging
192
193     calibrateAccelerometer(); // Calibrate ADXL345
194     startupCheck(); // Run hardware checks
195
196     motorSpeedPercentage = 25; // Set initial speed
197     setDcMotorSpeed();
198 }
199
200 // ----- Arduino Main Loop -----
201 /*
202  * Main control loop: handles mode switching, control, display, safety, and diagnostics.
203  */
204 void loop(){

```

```

204     if(true){
205         setControlModeOnJoyStickButtonPress(); // Switch between control modes
206         controlSkateBoard();                    // Main control logic
207         setNumberToSevenSegmentDisplay(getScaledMotorSpeedF-
orDigitalDisplay(motorSpeedPercentage)); // Update 7-segment
208         setServoPosition(getScaledMotorSpeedForAnalogDisplay(motorSpeedPercentage));
// Update servo
209         handleFall();                            // Detect and handle falls
210         while(isMasterLocked){
211             // Blink display to indicate lock
212             digitalWrite(shiftRegisterEnable,HIGH);
213             delay(1000);
214             digitalWrite(shiftRegisterEnable,LOW);
215             delay(1000);
216         };
217         slowDcMotorOnObstacle();                 // Obstacle avoidance
218         setHeadLightLedBrightness();             // Headlight control
219         debugLog();                              // Print debug info
220     }
221     else {
222         // Test routines (not used in normal operation)
223         testHandleFall();
224         testHeadlight();
225         testJoyStick();
226         testSpeedDisplay();
227         testUltraSound();
228         testStepperMotor();
229     }
230 }
231
232 // ----- Sensor Calibration -----
233 /*
234  * Calibrates the ADXL345 accelerometer by setting offsets for each axis.
235  */
236 void calibrateAccelerometer(){
237     Wire.begin();
238     Wire.beginTransmission(ADXL345);
239     Wire.write(0x2D);
240     Wire.write(8);
241     Wire.endTransmission();
242
243     //X-axis
244     Wire.beginTransmission(ADXL345);
245     Wire.write(0x1E); // X-axis offset register
246     Wire.write(accelerometerXOffset);
247     Wire.endTransmission();
248     delay(10);
249
250     //Y-axis
251     Wire.beginTransmission(ADXL345);
252     Wire.write(0x1F); // Y-axis offset register
253     Wire.write(accelerometerYOffset);
254     Wire.endTransmission();
255     delay(10);

```

```

256
257
258 //Z-axis
259 Wire.beginTransaction(ADXL345);
260 Wire.write(0x20); // Z-axis offset register
261 Wire.write(accelerometerZOffset);
262 Wire.endTransmission();
263 delay(10);
264 }
265
266 // ----- Main Control Logic -----
267 /*
268  * Selects control mode: accelerometer or joystick.
269  */
270 void controlSkateBoard(){
271     switch (isRemoteControlled)
272     {
273     case 0:
274         // Serial.println("Accelerometer Control Mode");
275         onAccelerometerControl();
276         break;
277
278     case 1:
279         // Serial.println("Remote Control Mode");
280         onDroneControl();
281         break;
282     default:
283         // Serial.println("No control selected");
284         break;
285     }
286 }
287
288 // ----- Debug Logging -----
289 /*
290  * Prints current system state to Serial for debugging.
291  */
292 void debugLog(){
293     Serial.print(" |Speed ");
294     Serial.print(motorSpeed);
295     Serial.print("(");
296     Serial.print(motorSpeedPercentage);
297     Serial.print(")");
298     if(isRemoteControlled){
299         Serial.print(" |joyX ");
300         Serial.print(joyStickThumbGripXReading);
301         Serial.print(" |joyY: ");
302         Serial.print(joyStickThumbGripYReading);
303     }
304     else{
305         Serial.print(" |aX ");
306         Serial.print(accelerationX);
307         Serial.print(" |aY ");
308         Serial.print(accelerationY);
309         Serial.print(" |aZ ");

```

```

310     Serial.print(accelerationZ);
311 }
312 Serial.print(" |stepPos");
313 Serial.print(lastStepperMotorPosition);
314 Serial.print(" |dist ");
315 Serial.print(getUltraSonicDistance());
316 Serial.print(" |fall ");
317 Serial.print(isMasterLocked);
318 Serial.print(" |totalG ");
319 Serial.println(getTotalAcceleration(accelerationX, accelerationY, accelerationZ));
320 }
321
322 // ----- Sensor Reading Functions -----
323 /*
324  * Reads acceleration values from ADXL345.
325  */
326 void getAccelerations(){
327     Wire.beginTransmission(ADXL345);
328     Wire.write(0x32);
329     Wire.endTransmission(false);
330     Wire.requestFrom(ADXL345, 6 ,true);
331     accelerationX = (Wire.read() | Wire.read() << 8);
332     accelerationX = accelerationX/256;
333     accelerationY = (Wire.read() | Wire.read() << 8);
334     accelerationY = accelerationY/256;
335     accelerationZ = (Wire.read() | Wire.read() << 8);
336     accelerationZ = accelerationZ/256;
337 }
338
339 /*
340  * Reads joystick X and Y analog values with debounce.
341  */
342 void getJoyStickThumbGripReadings(){
343     if((millis() - joyStickThumbGripXDebounceTime) > joyStickThumbGripDebounceDelay){
344         joyStickThumbGripXDebounceTime = millis();
345         joyStickThumbGripXReading = analogRead(joyStickThumbGripX);
346     }
347     if((millis() - joyStickThumbGripYDebounceTime) > joyStickThumbGripDebounceDelay){
348         joyStickThumbGripYDebounceTime = millis();
349         joyStickThumbGripYReading = analogRead(joyStickThumbGripY);
350     }
351 }
352
353 // ----- Display Scaling Functions -----
354 /*
355  * Scales motor speed percentage for 7-segment digital display (0-9).
356  */
357 int getScaledMotorSpeedForDigitalDisplay(int motorSpeedPercentage){
358     int scaledSpeed = motorSpeedPercentage >2 && motorSpeedPercentage <19 ? 1 :
static_cast<int>(motorSpeedPercentage)/10;
359     return scaledSpeed == 10? 9 : scaledSpeed;
360 }
361
362 /*

```

```

363  * Scales motor speed percentage for analog servo display (0-180 degrees).
364  */
365  int getScaledMotorSpeedForAnalogDisplay(int motorSpeedPercentage){
366      // Might need to return 180- if direction is in reverse
367      // return 180*(static_cast<int>(motorSpeed)/255.0);
368      return map(static_cast<int>(motorSpeedPercentage), 0, 100, 0, 180);
369  }
370
371  // ----- Utility Functions -----
372  /*
373   * Calculates total acceleration magnitude (g-force).
374   */
375  float getTotalAcceleration(float accelerationX, float accelerationY, float accelerationZ)
376  {
377      return sqrt(accelerationX*accelerationX + accelerationY*accelerationY +
378      accelerationZ*accelerationZ);
379  }
380  /*
381   * Measures distance to obstacle using ultrasonic sensor (in cm).
382   */
383  float getUltraSonicDistance(){
384      digitalWrite(ultraSonicTrigger, LOW);
385      delayMicroseconds(2);
386      digitalWrite(ultraSonicTrigger, HIGH);
387      delayMicroseconds(10);
388      digitalWrite(ultraSonicTrigger, LOW);
389      float duration = pulseIn(ultraSonicEcho, HIGH);
390      const float distance = (duration * 0.0343)/2;
391      return distance;
392  }
393  /*
394   * Detects a fall based on acceleration magnitude.
395   */
396  bool isFallDetected(float accelerationX, float accelerationY, float accelerationZ) {
397      float accelerationMagnitude = getTotalAcceleration(accelerationX, accelerationY,
398      accelerationZ);
399      return (accelerationMagnitude < gForceDeadzoneMinimum) || (accelerationMagnitude >
400      gForceDeadzoneMaximum);
401  }
402  // ----- Safety and Fall Handling -----
403  /*
404   * Locks system and stops motor if a fall is detected.
405   */
406  void handleFall(){
407      getAccelerations();
408      if(isFallDetected(accelerationX, accelerationY, accelerationZ)){
409          isMasterLocked = true;
410          motorSpeed = 0;
411          digitalWrite(dcMotorOutput, LOW);
412          headLightBrightness = 255;
413          setHeadLightLedBrightness();

```



```

413     }
414 }
415
416 // ----- Control Modes -----
417 /*
418  * Accelerometer-based speed control.
419  */
420 void onAccelerometerControl(){
421     getAccelerations();
422     if(fabs(accelerationX)>accelerationXThreshold){
423         int speedIncrement = minimumSpeedIncrement + ((fabs(accelerationX)-
accelerationXThreshold)/(accelerationXMaximum - accelerationXThreshold))*
(maximumSpeedIncrement - minimumSpeedIncrement);
424         if(accelerationX>0){
425             motorSpeedPercentage = motorSpeedPercentage + speedIncrement;
426         }
427         else if (accelerationX <0){
428             motorSpeedPercentage = motorSpeedPercentage - speedIncrement;
429         }
430         motorSpeedPercentage = constrain(motorSpeedPercentage, 0 ,100);
431         setDcMotorSpeed();
432     }
433 }
434
435 /*
436  * Joystick-based speed and steering control.
437  */
438 void onDroneControl(){
439     getJoyStickThumbGripReadings();
440     // motorSpeed = ((1024-joyStickThumbGripXReading)/1024.0)*255;
441     long scaledJoyStickThumbGripXReading = map(joyStickThumbGripXReading,0,1023,-512,512);
442     if(fabs(scaledJoyStickThumbGripXReading) > joyStickThumbGripXThreshold){
443         int speedIncrement = map(fabs(scaledJoyStickThumbGripXReading), 0, 512,
minimumSpeedIncrement, maximumSpeedIncrement);
444         speedIncrement = constrain(speedIncrement, minimumSpeedIncrement,
maximumSpeedIncrement);
445         if(scaledJoyStickThumbGripXReading<0){
446             motorSpeedPercentage = motorSpeedPercentage - speedIncrement;
447         } else if (scaledJoyStickThumbGripXReading > 0)
448         {
449             motorSpeedPercentage = motorSpeedPercentage + speedIncrement;
450         }
451         motorSpeedPercentage = constrain(motorSpeedPercentage, 0 ,100);
452         setDcMotorSpeed();
453     }
454
455     int targetPosition = map(joyStickThumbGripYReading, 0, 1023, -180, 180);
456     targetPosition = constrain(targetPosition, -180, 180);
457     int stepsToMove = targetPosition - lastStepperMotorPosition;
458     stepperMotor.setSpeed(10);
459     stepperMotor.step(stepsToMove);
460     lastStepperMotorPosition = targetPosition;
461
462 }

```

```

463
464 // ----- Mode Switching -----
465 /*
466  * Toggles control mode on joystick button press (with debounce).
467  */
468 bool setControlModeOnJoyStickButtonPress(){
469     bool currentJoyStickButtonState = digitalRead(joystickButton);
470     if(currentJoyStickButtonState != lastJoyStickButtonState && currentJoyStickButtonState
== 0){
471         if((millis() - joyStickButtonDebounceTime) > joyStickButtonDebounceDelay){
472             isRemoteControlled = !isRemoteControlled;
473             joyStickButtonDebounceTime = millis();
474         }
475     }
476     lastJoyStickButtonState = currentJoyStickButtonState;
477 }
478
479 // ----- Motor and Output Control -----
480 /*
481  * Sets DC motor speed based on current speed percentage and lock state.
482  */
483 void setDcMotorSpeed() {
484     if(isMasterLocked){
485         motorSpeed = 0;
486     } else {
487         motorSpeed = map(motorSpeedPercentage, 0, 100,
minimumMotorSpeed,maximumMotorSpeed);
488         motorSpeed = constrain(motorSpeed, 155, 255);
489     }
490     motorSpeedPercentage > 5 ? analogWrite(dcMotorOutput, motorSpeed) :
analogWrite(dcMotorOutput, 0);
491 }
492
493 /*
494  * Sets headlight LED brightness based on potentiometer or photoresistor.
495  */
496 void setHeadLightLedBrightness(){
497     int headLightResistanceReading;
498     if((millis() - headLightLedPotentiometerDebounceTime) > headLightLedPotentio-
meterDelay){
499         headLightLedPotentiometerDebounceTime = millis();
500         headLightResistanceReading = analogRead(headLightPotentiometer);
501         if(headLightResistanceReading < 100){
502             headLightResistanceReading = analogRead(headPhotoResistor);
503             // headLightBrightness = (255*(1023.0-headLightResistanceReading)/1024);
504             headLightBrightness = map(1023-headLightResistanceReading,0,1023,0,255);
505             headLightBrightness = constrain(headLightBrightness, 0,255);
506             Serial.print(" |PhoRes :");
507             Serial.print(headLightResistanceReading);
508         }
509         else{
510             // headLightBrightness = (255*(headLightResistanceReading)/1024);
511             headLightBrightness = map(headLightResistanceReading,0,1023,0,255);
512             headLightBrightness = constrain(headLightBrightness, 0,255);

```

```

513         Serial.print(" |PotRes");
514         Serial.print(headLightResistanceReading);
515     }
516     analogWrite(shiftRegisterEnable, headLightBrightness);
517 }
518 }
519
520 /*
521  * Updates 7-segment display with given digit.
522  */
523 void setNumberToSevenSegmentDisplay(int n){
524     digitalWrite(shiftRegisterOutputRegisterClock, LOW);
525     for(int i=7; i>=0; i--){
526         digitalWrite(shiftRegisterShiftClock, LOW);
527         digitalWrite(shiftRegisterInput, sevenSegmentDisplayNumberMap[n]
[registerOutputSevenSegmentDisplayMap[i]]);
528         digitalWrite(shiftRegisterShiftClock, HIGH);
529     }
530     digitalWrite(shiftRegisterShiftClock, LOW);
531     digitalWrite(shiftRegisterInput, headlightLED);
532     digitalWrite(shiftRegisterShiftClock, HIGH);
533     digitalWrite(shiftRegisterOutputRegisterClock, HIGH);
534 }
535
536 /*
537  * Sets servo position for analog speed display.
538  */
539 void setServoPosition(int servoPosition){
540     speedometerServo.write(servoPosition);
541 }
542
543 /*
544  * Slows down DC motor if obstacle detected by ultrasonic sensor.
545  */
546 void slowDcMotorOnObstacle(){
547     int distanceToObstacle = getUltraSonicDistance();
548     if (distanceToObstacle< ultraSoundObstacleDistanceThreshold){
549         motorSpeedPercentage = map(distanceToObstacle, emergencyBrakeThreshold,
ultraSoundObstacleDistanceThreshold, 0, 100)/brakingFactor;
550         // motorSpeed = ((distanceToObstacle/ultraSoundObstacleDi-
stanceThreshold)*255)/brakingFactor;
551         // motorSpeedPercentage = distanceToObstacle < emergencyBrakeThreshold ? 0 :
motorSpeedPercentage;
552         motorSpeedPercentage = constrain(motorSpeedPercentage, 0, preObstacleSpeedPerc-
centage/brakingFactor);
553         setDcMotorSpeed();
554     } else
555     {
556         preObstacleSpeedPercentage = motorSpeedPercentage;
557     }
558
559 }
560
561 // ----- Startup and Test Routines -----

```

```

562  /*
563  * Runs hardware checks at startup: servo, display, motor, sensors.
564  */
565  void startupCheck(){
566      // Servo check
567      setServoPosition(180);
568      delay(1000);
569      setServoPosition(0);
570
571      // 7 Segment display check
572      // Headlight LED check
573      for(int i=0; i<=9; i++){
574          headlightLED = i%2;
575          setNumberToSevenSegmentDisplay(i);
576          delay(250);
577      }
578
579      // motor check
580      motorSpeedPercentage = 20;
581      setDcMotorSpeed();
582      delay(700);
583      motorSpeedPercentage = 50;
584      setDcMotorSpeed();
585      delay(700);
586      motorSpeedPercentage = 100;
587      setDcMotorSpeed();
588
589      // Ultrasound check
590      Serial.print("Ultrasonic distance: ");
591      Serial.println(getUltraSonicDistance());
592
593      // Joystick check
594      Serial.print("Button state: ");
595      Serial.print(digitalRead(joystickButton));
596      Serial.print(" | Stick X Reading: ");
597      Serial.print(analogRead(joyStickThumbGripX));
598      Serial.print(" | Stick Y Reading: ");
599      Serial.print(analogRead(joyStickThumbGripY));
600
601      // Stepper check
602      testStepperMotor();
603  }
604
605  /*
606  * Test routines for debugging and validation.
607  */
608  void testHandleFall(){
609      handleFall();
610  }
611
612  void testHeadlight(){
613      setHeadLightLedBrightness();
614  }
615

```

```

616 void testJoyStick(){
617     setControlModeOnJoyStickButtonPress();
618     Serial.print("Control mode: ");
619     Serial.println(isRemoteControlled);
620     controlSkateBoard();
621     delay(200);
622 }
623
624 void testSpeedDisplay(){
625     for(int i=0; i<255; i++){
626         motorSpeed = i;
627         setDcMotorSpeed();
628         setNumberToSevenSegmentDisplay(getScaledMotorSpeedForDigitalDisplay(i));
629         setServoPosition(getScaledMotorSpeedForAnalogDisplay(i));
630         Serial.print(getScaledMotorSpeedForDigitalDisplay(i));
631         Serial.print(" ");
632         Serial.println(getScaledMotorSpeedForAnalogDisplay(i));
633         delay(700);
634     }
635 }
636
637 void testUltraSound(){
638     motorSpeed = 255;
639     setDcMotorSpeed();
640     slowDcMotorOnObstacle();
641 }
642
643 void testStepperMotor(){
644     getJoyStickThumbGripReadings();
645     if((millis() - joyStickThumbGripYDebounceTime) > joyStickThumbGripDebounceDelay){
646         long scaledY = map(joyStickThumbGripYReading, 0, 1023, -512, 512);
647         long targetPosition = constrain(scaledY, -180, 180);
648         long stepsToMove = targetPosition - lastStepperMotorPosition;
649         Serial.print(targetPosition);
650         stepperMotor.setSpeed(10); // You can make speed dynamic if needed
651         stepperMotor.step(stepsToMove);
652         lastStepperMotorPosition = targetPosition;
653         joyStickThumbGripYDebounceTime = millis();
654     }
655     Serial.println(" |");
656 }
657

```